FITFLOW: TECHNICAL SYSTEM MANUAL

A Comprehensive System Walkthrough, Architectural Blueprint, & Viva Preparation Guide

Candidate Name:	ABDALLA HASHIM AHMED ABDALLA (B032320119)
Supervisor:	Ts. Muhammad Faheem Bin Mohd Ezani
Faculty:	Faculty of Information and Communication Technology (FTMK)
Academic Program:	Bachelor of Computer Science (Software Development) (BITS) with Honours
Academic Year:	2026/2027 (Semester 5 - PSM 1 Submission)

Purpose: This document outlines the technical underpinnings, libraries, APIs, and data structures utilized in the FitFlow portal. It is designed to prepare the candidate for technical questions raised during the Final Year Project (PSM 1) viva examination.

1. System Architecture & High-Level Flow

FitFlow is built using a modern decoupled architecture consisting of a React client, a FastAPI service gateway, and a cloud-based Supabase relational backend database:

- **Frontend Layer (React.js + Vite):** Handles UI rendering, user session states, drag-and-drop Kanban tracking, and local audio streaming via the browser's Web Speech API.
- **Backend Layer (FastAPI + Python):** Serves as the AI microservice gateway, coordinating streaming speech requests with Gemini and OpenAI APIs.
- **Database Layer (Supabase PostgreSQL):** Directly processes data transactions, enforces Row-Level Security (RLS) tables, and computes cosine similarity scores via database-level SQL Stored Procedures.

2. Web Scraping & Data Aggregation Module

To address **Search Fragmentation (Problem 1)**, FitFlow utilizes a scheduled crawler to gather vacancy data:

Technical Stack & Python Libraries:

- **Selenium WebDriver:** Used for javascript-heavy dynamic job portals (e.g. JobStreet, LinkedIn) to trigger infinite scrolls and bypass initial authorization scripts. Runs headlessly on GitHub Actions runner environments.
- **BeautifulSoup4 (bs4):** Used to parse the raw HTML structure once loaded, extracting job title, company, description, salary, location, and required technical tags.
- **Requests:** Utilized for REST API endpoints or static job sources that do not require dynamic DOM execution (fast crawler fallback).
- **Supabase-py Client:** Used to batch-insert scraped items into the Supabase database.

Execution Workflow:

The scraper is hosted within the GitHub repository and executed via a **GitHub Actions Workflow** scheduled to run daily at midnight via a cron expression: 0 0 * * *. Once scraping finishes, a database-level trigger automatically re-runs student profile recommendation score calculations.

3. Profiling, Onboarding & Recommendation Engine

To address **Skill-to-Role Misalignment (Problem 2)**, FitFlow profiles students during onboarding and executes a multi-criteria compatibility score:

The Matching Algorithm Formula:

The recommendation score is calculated via the weighted formula:

$$Match\ Score = (0.75 \times Skill\ Match) + (0.15 \times Preferred\ Role\ Match) + (0.10 \times Programme\ Relevance)$$

- **Skill Match (75% Weight):** Calculated using *Cosine Similarity*. The student's skills and the internship's required skills are modeled as binary vectors. The match score is computed directly as the intersection of student skills and required vacancy skills, divided by the total required skills.
- **Preferred Role Match (15% Weight):** A binary check (1 or 0) evaluating whether the vacancy's categorized role matches the career preferences declared by the student during onboarding.
- **Programme Relevance (10% Weight):** Evaluates whether the internship aligns with the student's FTMK academic program specialization (e.g., Software Development BITS, Computer Security BITG) based on a pre-defined relevance weight table.

Why Database-Level PL/pgSQL Stored Procedures?

Rather than fetching thousands of scraped internships to the React frontend or FastAPI server to compute matches in Python (which incurs high latency and network overhead), the algorithm is implemented as a **PostgreSQL Stored Procedure (PL/pgSQL)** directly in Supabase. The function executes natively on database updates, returning pre-calculated ranked lists in milliseconds.

4. Kanban Tracker State Machine

To address **Application tracking difficulties (Problem 3)**, FitFlow implements an interactive Kanban tracker:

- **Frontend Implementation:** Uses react-beautiful-dnd or HTML5 Drag and Drop API. Tracks user interactions and maps drag transitions to state changes.
- **State Machine Stages:** Application cards move sequentially through: SAVED → APPLIED → INTERVIEWING → OFFERED → REJECTED.
- **Real-time Sync:** Triggers immediate REST updates to the tracker_applications table in Supabase via RLS secure endpoints, keeping database records up-to-date.

5. AI Mock Interview Prep Module (Detailed Voice Pipeline)

This module is the core preparation feature of FitFlow. It enables students to conduct mock verbal interviews based directly on their recommended vacancies:

The Audio Voice Room Pipeline:

The interaction in the interview room runs entirely client-side without heavy media server infrastructure by using native web APIs:

- **Text-to-Speech (TTS) vocalization:** Uses the browser's native **Web Speech API SpeechSynthesis** interface. When a question is generated, the system calls window.speechSynthesis.speak(new SpeechSynthesisUtterance(question)) to vocalize it.
- **Speech-to-Text (STT) transcription:** Uses the native **Web Speech API SpeechRecognition** interface (webkit prefix in Chrome). When the user clicks the microphone button, the speech recognizer captures microphone audio and transcribes it in real-time.
- **Audio Waveform Visualization:** Uses the **Web Audio API**. The client captures the media stream, pipes it through an AnalyserNode, and extracts frequency data to animate the dynamic SVG waveform.

The AI Dialogue & Evaluation Pipeline (FastAPI Backend):

The backend coordinates the generative AI services:

- **Real-time Dialogue Agent (Gemini 1.5 Flash API):** Generates context-aware, role-specific follow-up questions. FastAPI sends the job description and the dialogue turn history to the Gemini API, which generates the next conversational question.
- **Multi-Dimensional Rubric Grading (OpenAI GPT-4o API):** Upon interview completion, the conversation transcript is sent to `/interviews/evaluate`. GPT-4o evaluates the candidate across four UTeM-aligned rubrics: Technical Competency (knowledge accuracy), Communication (structure, vocabulary), Behavioral (STAR method alignment), and Confidence (tonal assurance). It outputs structured JSON containing grades, qualitative rationale, and identified skill gaps.
- **Skill Gaps Choose Platform Redirect:** The identified skill gaps are saved to `interview_skill_gaps`. When clicked, a modal overlay queries the vacancies table to see how many matching roles are unlocked by mastering that skill and redirects the student to Udemy, Coursera, or YouTube with a pre-filled search query.

6. Common Viva / Examiner Q&A; Cheatsheet

Prepare for these common technical questions during your evaluation:

Q: Why did you choose Supabase over a standard node/express + MongoDB stack?

A: Supabase provides a PostgreSQL database natively, which is vital for executing relational math functions like Cosine Similarity via SQL Stored Procedures. It also includes built-in Row-Level Security (RLS) policies, built-in Authentication APIs, and instant REST capabilities, reducing backend boilerplate so we could focus on the AI pipelines.

Q: Explain how the Speech-to-Text works. Do you pay for an API?

A: No. We utilize the browser's native Web Speech API (SpeechRecognition). Since the processing runs client-side within the browser engine, it is entirely free, fast, and does not incur any network latency or transcription costs on our FastAPI server.

Q: Why do you have two distinct LLMs (Gemini and OpenAI) in the backend?

A: We optimize for cost, speed, and analytical quality. Gemini 1.5 Flash is used for the dialogue agent because it is extremely fast (low latency is critical during live speech loops) and cost-effective. OpenAI GPT-4o is used for post-session grading because it excels at complex evaluation, structured JSON schema output, and qualitative rubric analysis.

Q: What is database normalization, and how is it applied in your tables?

A: Database normalization organizers tables to eliminate redundancy and prevent anomalies. In FitFlow, all tables are normalized to 3NF. For example, student details are separated from credentials (profiles 1:1 student_profiles), many-to-many relationships (students-to-skills and vacancies-to-skills) are resolved using composite-key junction tables (user_skills, internship_skills), and the AI module stores transcripts, evaluations, and skill gaps in four distinct tables linked by UUID foreign keys.

Q: How does the Choose Platform redirect dialog help the student?

A: It bridges the gap between evaluation and remediation. Rather than just telling the student they failed a skill, the redirect dialog queries our database to show how many matching vacancies require that skill. It then provides direct links to search queries on Udemy, Coursera, and YouTube, enabling immediate learning.